



MVP – Engenharia de Dados

Panorama comercial de uma operação de e-commerce

Aluno: Daniel Ulhoa Oliveira Miranda de Siqueira

Email: danielulhoa@gmail.com

GitHub: <https://github.com/danielulhoa-wq>

Sprint: Engenharia de Dados (40530010057_20260_01)

Panorama comercial de uma operação de e-commerce.....	1
Contextualização e objetivos gerais.....	3
Detalhamento do projeto.....	3
Base de Dados e ferramentas selecionadas.....	3
Coleta e carregamento de dados.....	4
Tratamento / Modelagem de dados.....	4
Catálogo de dados.....	9
Análise de dados.....	9
Qualidade de dados.....	9
Análise final dos dados (Perguntas + Respostas).....	13
Considerações finais.....	17
Pontos de melhoria.....	17
Autoavaliação.....	18

Contextualização e objetivos gerais

É essencial compreender as principais características comerciais de um negócio para tomar decisões que sustentem o crescimento das vendas, mas nem sempre essa visão está disponível de forma clara e acessível. Este trabalho tem como propósito construir um pipeline de dados capaz de transformar registros brutos de transações de um e-commerce em um panorama comercial objetivo.

Desta forma, serão buscadas as respostas para as seguintes perguntas:

- 1) Quais são os produtos mais vendidos em quantidade e por faturamento?
- 2) Quais regiões concentram as vendas em volume e por faturamento?
- 3) Quais são os maiores clientes em termos de faturamento?

Detalhamento do projeto

Base de Dados

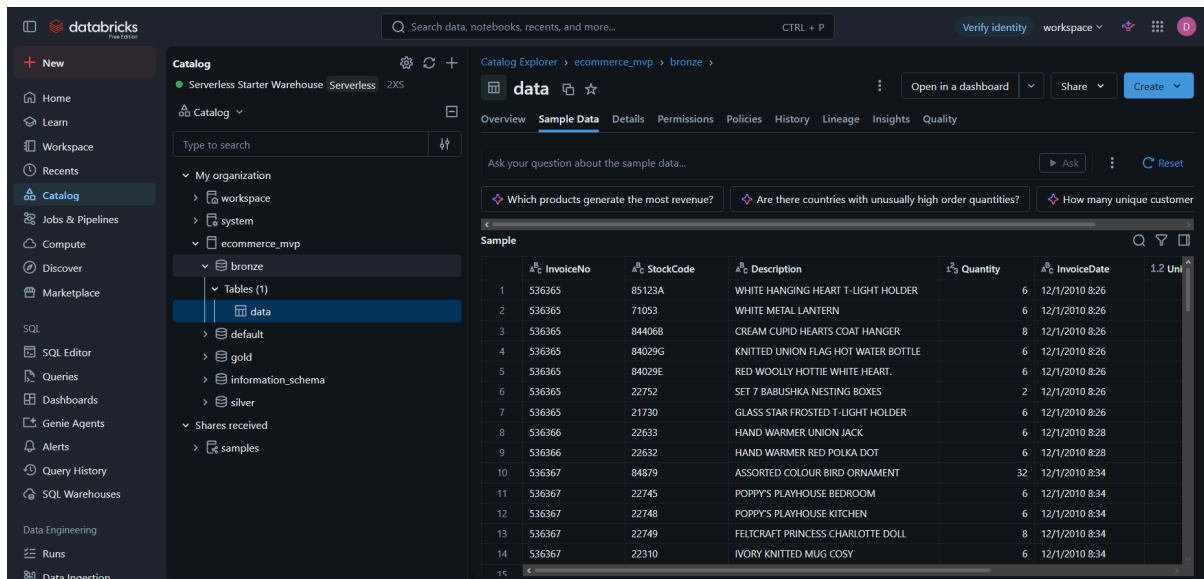
- [E-commerce Data](#): documento público contendo pedidos de um e-commerce, contendo informações de dezembro de 2010 até dezembro de 2011.

Ferramentas utilizadas:

- Databricks (Free Edition): utilizado para o desenvolvimento completo do pipeline. Dentro da plataforma foram exploradas a linguagem SQL e a API PySpark para realização do ETL nas camadas bronze, prata e ouro.
- GitHub: utilizado como repositório e entrega final dos documentos utilizados e o projeto final.
- brModelo: utilizado para a representação visual do modelo de dados relacional (esquema estrela), incluindo entidades, atributos e cardinalidades entre a tabela fato e as dimensões.

Coleta e carregamento de dados

O arquivo utilizado foi baixado diretamente em seu link de acesso público no [kaggle](#), em formato CSV, e por ser um documento único, não foi necessário realizar junções de tabelas. Ele foi carregado diretamente no schema bronze do *catalog* "ecommerce_mvp", ambos criados previamente junto das demais camadas silver e gold.



	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice
1	536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	12/1/2010 8:26	
2	536365	71053	WHITE METAL LANTERN	6	12/1/2010 8:26	
3	536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	12/1/2010 8:26	
4	536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	12/1/2010 8:26	
5	536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	12/1/2010 8:26	
6	536365	22752	SET 7 BABUSHKA NESTING BOXES	2	12/1/2010 8:26	
7	536365	21730	GLASS STAR FROSTED T-LIGHT HOLDER	6	12/1/2010 8:26	
8	536366	22633	HAND WARMER UNION JACK	6	12/1/2010 8:28	
9	536366	22632	HAND WARMER RED POLKA DOT	6	12/1/2010 8:28	
10	536367	84879	ASSORTED COLOUR BIRD ORNAMENT	32	12/1/2010 8:34	
11	536367	22745	POPPY'S PLAYHOUSE BEDROOM	6	12/1/2010 8:34	
12	536367	22748	POPPY'S PLAYHOUSE KITCHEN	6	12/1/2010 8:34	
13	536367	22749	FELTCRAFT PRINCESS CHARLOTTE DOLL	8	12/1/2010 8:34	
14	536367	22310	IVORY KNITTED MUG COSY	6	12/1/2010 8:34	
15						

Figura 1 - Tabela de dados carregada no *schema* bronze do Databricks

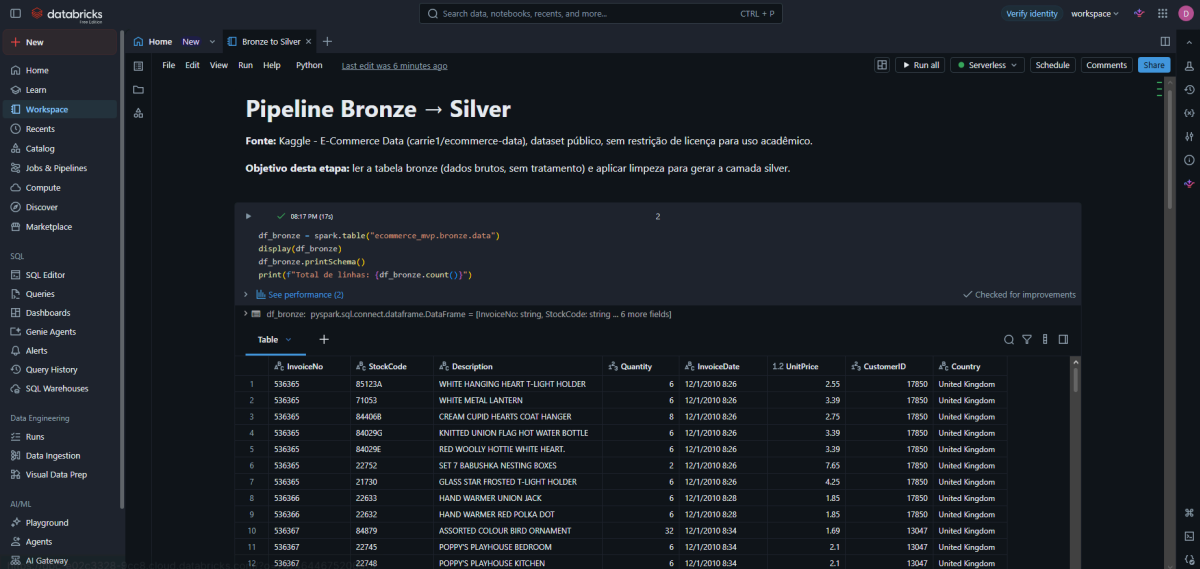
Tratamento de Dados

Considerando as perguntas estabelecidas como multidimensionais (produto, região e cliente) e buscando realizar consultas de forma pragmática e eficiente, optou-se pelo esquema estrela como modelo dos dados ao invés do floco de neve, cuja normalização adicional das dimensões não teria um ganho relevante ao projeto dado o baixo volume de atributos por dimensão.

O processo de ETL foi ramificado em dois notebooks distintos, um por transição de camada da arquitetura medalhão — "Bronze to Silver", responsável pela limpeza e tratamento dos

dados brutos, e "Silver to Gold", responsável pela construção do modelo dimensional e das tabelas de resposta às perguntas do MVP — em vez de consolidado em um único notebook.

Camada Bronze -> Silver



Pipeline Bronze → Silver

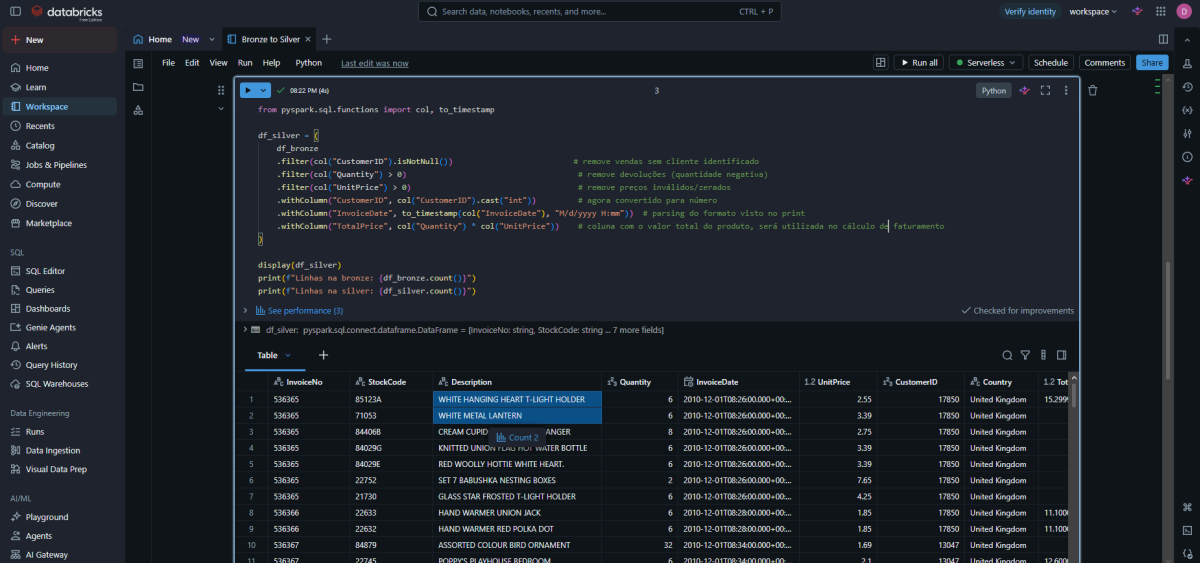
Fonte: Kaggle - E-Commerce Data (carrie1/ecommerce-data), dataset público, sem restrição de licença para uso acadêmico.

Objetivo desta etapa: ler a tabela bronze (dados brutos, sem tratamento) e aplicar limpeza para gerar a camada silver.

```
df_bronze = spark.table("ecommerce_mv_bronze_data")
display(df_bronze)
df_bronze.printSchema()
print(f"Total de linhas: {df_bronze.count()}")
```

InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	12/1/2010 8:26	2.55	17850	United Kingdom
536365	71053	WHITE METAL LANTERN	6	12/1/2010 8:26	3.39	17850	United Kingdom
536365	844068	CREAM CUPID HEARTS COAT HANGER	8	12/1/2010 8:26	2.75	17850	United Kingdom
536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	12/1/2010 8:26	3.39	17850	United Kingdom
536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	12/1/2010 8:26	3.39	17850	United Kingdom
536365	22752	SET 7 BABUSHKA NESTING BOXES	2	12/1/2010 8:26	7.65	17850	United Kingdom
536365	21730	GLASS STAR FROSTED T-LIGHT HOLDER	6	12/1/2010 8:26	4.25	17850	United Kingdom
536366	22633	HAND WARMER UNION JACK	6	12/1/2010 8:28	1.85	17850	United Kingdom
536366	22632	HAND WARMER RED POLKA DOT	6	12/1/2010 8:28	1.85	17850	United Kingdom
536367	84879	ASSORTED COLOUR BIRD ORNAMENT	32	12/1/2010 8:34	1.69	13047	United Kingdom
536367	22745	POPPY'S PLAYHOUSE BEDROOM	6	12/1/2010 8:34	2.1	13047	United Kingdom
536367	22748	POPPY'S PLAYHOUSE KITCHEN	6	12/1/2010 8:34	2.1	13047	United Kingdom

Figura 2 - Notebook para tratamento dos dados brutos (camada bronze) parte 1 (disponível em [bronze_to_silver.ipynb](#))



```
from pyspark.sql.functions import col, to_timestamp

df_silver = df_bronze
    .filter(col("CustomerID").isNotNull())
    .filter(col("Quantity") > 0)
    .filter(col("UnitPrice") > 0)
    .withColumn("CustomerID", col("CustomerID").cast("int"))
    .withColumn("InvoiceDate", to_timestamp(col("InvoiceDate"), "M/d/yyyy H:mm"))
    .withColumn("TotalPrice", col("Quantity") * col("UnitPrice"))

display(df_silver)
print(f"Linhas na bronze: {df_bronze.count()}")
print(f"Linhas na silver: {df_silver.count()}")
```

InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country	TotalPrice
536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	2010-12-01T08:26:00.000-00:00	2.55	17850	United Kingdom	15.299
536365	71053	WHITE METAL LANTERN	6	2010-12-01T08:26:00.000-00:00	3.39	17850	United Kingdom	
536365	844068	CREAM CUPID HEARTS COAT HANGER	8	2010-12-01T08:26:00.000-00:00	2.75	17850	United Kingdom	
536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	2010-12-01T08:26:00.000-00:00	3.39	17850	United Kingdom	
536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	2010-12-01T08:26:00.000-00:00	3.39	17850	United Kingdom	
536365	22752	SET 7 BABUSHKA NESTING BOXES	2	2010-12-01T08:26:00.000-00:00	7.65	17850	United Kingdom	
536365	21730	GLASS STAR FROSTED T-LIGHT HOLDER	6	2010-12-01T08:26:00.000-00:00	4.25	17850	United Kingdom	
536366	22633	HAND WARMER UNION JACK	6	2010-12-01T08:28:00.000-00:00	1.85	17850	United Kingdom	11.100
536366	22632	HAND WARMER RED POLKA DOT	6	2010-12-01T08:28:00.000-00:00	1.85	17850	United Kingdom	11.100
536367	84879	ASSORTED COLOUR BIRD ORNAMENT	32	2010-12-01T08:34:00.000-00:00	1.69	13047	United Kingdom	
536367	22745	POPPY'S PLAYHOUSE BEDROOM	6	2010-12-01T08:34:00.000-00:00	2.1	13047	United Kingdom	12.600

Figura 3 - Notebook para tratamento dos dados brutos (camada bronze para silver) parte 2

Após a remoção de linhas sem `CustomerID`, devoluções (`quantity < 0`) e preços inválidos, foi observada uma redução de 26,58% na quantidade de linhas do documento (541.909 → 397.884).

Além dessas alterações, também foi feito:

- Criação de uma nova coluna ("`TotalPrice`") para calcular o valor total de cada item do pedido.
- `InvoiceDate` convertido para *timestamp*.
- `CustomerID` convertido para inteiro.

Camada Silver -> Gold

Nesta etapa, disponível em `silver_to_gold.ipynb`, houve a construção das dimensões abaixo:

- `dim_produto`: `StockCode` (chave) + `description`
- `dim_cliente`: `CustomerId` (chave)
- `dim_regiao`: `country` (chave)
- `dim_tempo`: `InvoiceDate` (chave) + `Ano` + `Mes` + `dia`

Também foi criada a tabela fato (`fato_vendas`) e uma tabela derivada, `agg_pedidos`. Diferente da tabela fato, cujo grão é o item vendido (uma linha por produto dentro de um pedido), `agg_pedidos` possui grão de pedido completo — obtida pela agregação de `fato_vendas` por `InvoiceNo`, somando o valor de todos os itens de cada nota fiscal em um único valor total (`ValorTotalPedido`). Essa distinção de grão é necessária porque métricas como ticket médio exigem que cada pedido seja contado uma única vez; calculá-las diretamente sobre `fato_vendas` geraria distorção, já que pedidos com mais itens teriam peso desproporcional no cálculo. `agg_pedidos` foi utilizada como base para o ticket médio incorporado nas três tabelas de resposta às perguntas do MVP

Dando sequência no processo de melhoria de qualidade dos dados, foi observado que alguns Stock Codes (código identificador único do item) estavam referenciando mais de 1 rótulo de texto (provavelmente por alteração no nome do item, falta de padronização ou erro de digitação) o que não deveria ocorrer.

Assim, foi feito o ajuste na dimensão `dim_produto`, através da técnica em Spark “top-N por grupo via window function”, onde foi estabelecido um descritivo padrão para esses códigos e chegando numa relação 1(`StockCode`) para 1 (rótulo de texto).

Apenas para ilustrar brevemente a lógica da “top-N por grupo via window function”:

- Primeiro é necessário contar quantas vezes cada combinação de stock code + descritivo aparece nas vendas

- Depois é preciso separar os dados em grupos de stock code para ordenar os rótulos de texto mais frequentes, e atribuir um ranking de maior frequência sobre eles.
- Por fim, mantém-se apenas os rótulos de posição 1 no ranqueamento.

Por fim, foi feito um ajuste de tipos monetários no valor total dos pedidos pois foram identificados erros de arredondamento binários (Ex: 22.000000000000003)

Abaixo é possível visualizar a representação do modelo relacional (esquema estrela) através da ferramenta brModelo:

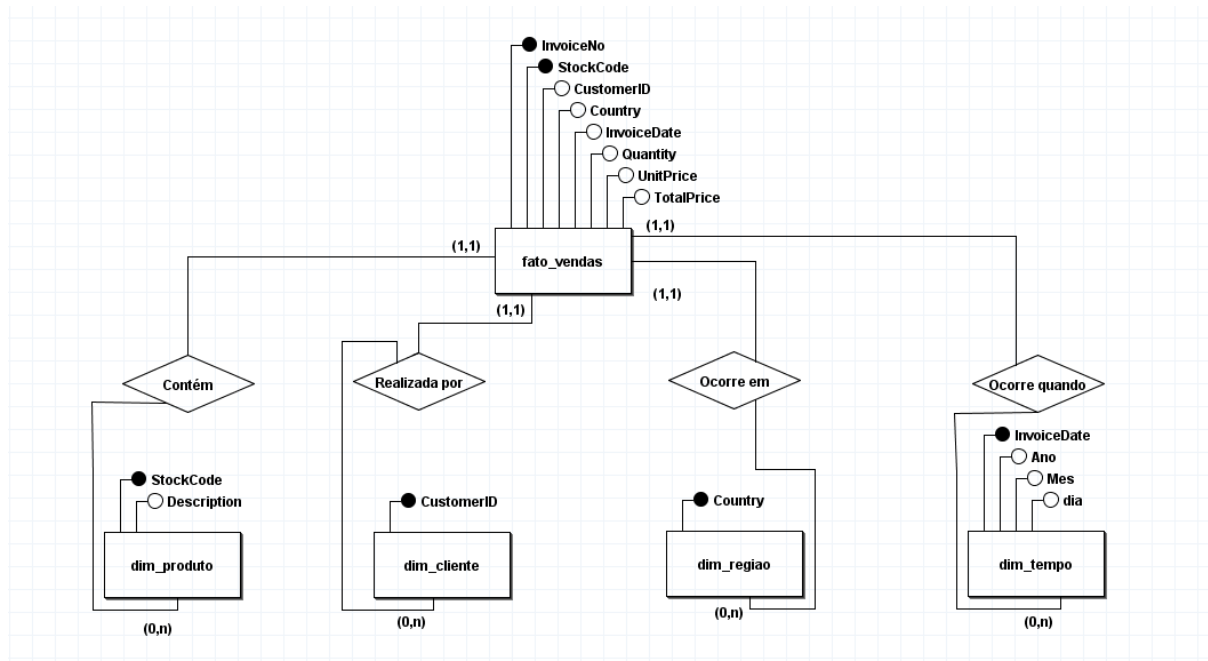


Figura 4 - Representação do modelo de dados relacional

Obs: A tabela agg_pedidos foi omitida do diagrama por ser uma tabela derivada, não uma dimensão.

Camadas de dados

- Bronze:

Catalog Explorer > ecommerce_mvp >

bronze   Use with BI tools Share Create

Overview Details Permissions Policies

Description 
Camada 1 da arquitetura medalhão

Filter tables

Tables 1 Volumes 0 Models 0 Functions 0 Secrets 0 Services 0 Skills 0 Connections 0

Sort

Name	Owner	Created at	Popularity	Comment
 data	danielulhoa@gmail.com	Aug 30, 2026, 08:28 PM		Created by the file uploa...

About this schema

Owner danielulhoa@gmail.com 

Figura 5 - Tabela bronze salva (persistida) na nuvem

- Silver:

Catalog Explorer > ecommerce_mvp >

silver   Use with BI tools Share Create

Overview Details Permissions Policies

Description 
camada 2 da arquitetura medalhão

Filter tables

Tables 1 Volumes 0 Models 0 Functions 0 Secrets 0 Services 0 Skills 0 Connections 0

Sort

Name	Owner	Created at	Popularity	Comment
 vendas_tratadas	danielulhoa@gmail.com	Aug 31, 2026, 08:24 PM		

Figura 6 - Tabela silver salva (persistida) na nuvem

- Gold:

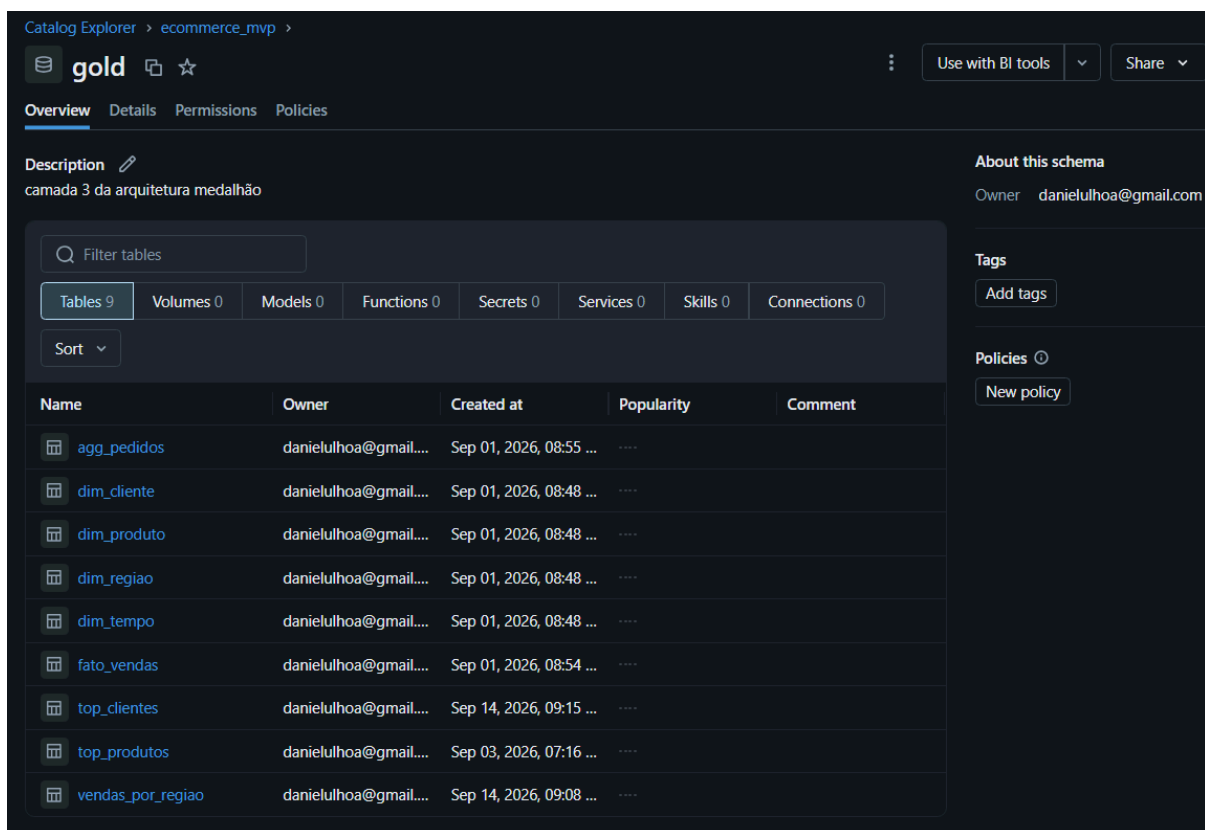


Figura 7 - Tabela gold salva (persistida) na nuvem

Catálogo de dados

1. Dados brutos (camada bronze)

Coluna	Tipo	Descrição
InvoiceNo	string	Identificador único do pedido/nota fiscal. Prefixo "C" indica pedido de cancelamento.
StockCode	string	Código do produto vendido. Também utilizado para lançamentos administrativos/financeiros (ex: "POST", "BANK CHARGES").
Description	string	Nome/descrição textual do produto associado ao StockCode.
Quantity	bigint	Quantidade de unidades vendidas do item. Valores negativos indicam devolução/cancelamento.
InvoiceDate	string	Data e hora da venda, no formato "M/d/yyyy H:mm".
UnitPrice	double	Preço unitário do item, na moeda original do dataset (libra esterlina).
CustomerID	string	Identificador do cliente que realizou a compra. Ausente em vendas não identificadas
Country	string	País onde a venda foi registrada.

Tabela 1 - Catálogo de dados brutos

Obs: InvoiceDate está como **string** (não como data/timestamp) nesta etapa de carga, porque a conversão de tipo só acontece depois, na camada silver — o catálogo aqui reflete fielmente o schema inferido no momento da ingestão do CSV, não o schema já tratado.

Análise de dados

A etapa de análise de dados será dividida em dois momentos, sendo o primeiro dedicado para análise de qualidade de dados e o segundo para responder às perguntas de negócio feitas na etapa de contextualização e objetivos do trabalho.

Qualidade de dados

A análise de qualidade foi conduzida sobre os dados no seu estado inicial de captura (camada bronze, 541.909 linhas) e seus códigos podem ser encontrados em silver_to_gold.ipynb, antes de qualquer tratamento. Foram avaliadas as cinco dimensões sugeridas: completude, consistência, unicidade, acurácia e outliers.

Na dimensão de **completude**, identificou-se que o atributo **CustomerID** apresenta 135.080 valores nulos, correspondendo a 24,93% do total — vendas realizadas sem identificação do cliente. O atributo **Description** apresenta uma taxa bem menor de nulos, 0,27% (1.454 registros). Os demais atributos não apresentaram valores nulos ou vazios. Os 1.454 registros (0,27%) com **Description** nula não foram tratados, pois o produto é identificado por **StockCode** (não por **Description**) na tabela fato — a ausência afeta apenas a legibilidade do nome, sem impactar quantidade ou faturamento calculados.

```
from pyspark.sql.functions import col, count, when

colunas = df_bronze.columns
tipos = dict(df_bronze.dtypes) # mapa coluna -> tipo (ex: 'string', 'bigint', 'double')

expressoes = []
for c in colunas:
    if tipos[c] == "string":
        expressoes.append(count.when(col(c).isNull() | (col(c) == ""), c).alias(c))
    else:
        expressoes.append(count.when(col(c).isNull(), c).alias(c))

completude = df_bronze.select(expressoes)
display(completude)

for c in colunas:
    qtd_nulos = completude.select(c).collect()[0][0]
    proporcao = (qtd_nulos / total_linhas) * 100
    print(f"{c}: {qtd_nulos} nulos/vazios ({proporcao:.2f}%)")
```

completude: pyspark.sql.connect.dataframe.DataFrame = [InvoiceNo: long, StockCode: long ... 6 more fields]

InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
1	0	0	1454	0	0	135080	0

Figura 8 - Análise de completude

Na dimensão de **consistência**, o atributo **InvoiceDate** mostrou-se 100% aderente ao padrão de data esperado (M/d/yyyy H:mm), sem nenhuma inconsistência de formato. Já o atributo **StockCode**, teoricamente esperado como um código numérico de 5 dígitos com letra opcional, apresentou 3.385 registros (0,62%) fora desse padrão. Foi feito um *deep dive*

desses casos que revelou que a maior parte não representa erro de captura, mas sim o uso do mesmo campo para lançamentos administrativos e financeiros. Alguns exemplos observados foram: postagem ("POST", 1.256 ocorrências), ajustes manuais ("M", 571 ocorrências), taxas bancárias ("BANK CHARGES") e taxas de marketplace ("AMAZONFEE"), que coexistem com códigos de produtos reais no mesmo atributo.

```

from pyspark.sql.functions import to_timestamp

# Tenta converter InvoiceDate no formato esperado; se falhar, vira nulo
# Isso revela quantos registros NÃO seguem o padrão "M/d/yyyy H:mm"
df_teste_data = df_bronze.withColumn(
    "data_convertida", to_timestamp(col("InvoiceDate"), "M/d/yyyy H:mm")
)

inconsistentes = df_teste_data.filter(col("data_convertida").isNull()).count()
print(f"Registros com InvoiceDate fora do padrão esperado: {inconsistentes} ((({inconsistentes/total_linhas}*100:.2f)%)")

> See performance (1)
df_teste_data: pyspark.sql.connect.dataframe.DataFrame = [InvoiceNo: string, StockCode: string ... 7 more fields]
Registros com InvoiceDate fora do padrão esperado: 0 (0.00%)

```

Figura 9 - Análise de consistência

```

display(
    df_teste_stockcode
    .filter(col("stockcode_valido") == False)
    .groupBy("StockCode")
    .count()
    .orderBy(col("count").desc())
)

> See performance (1)

```

StockCode	count
POST	1256
DOT	710
M	571
1505GBL	326
C2	144
D	77
S	63
1505GBL	62
BANK CHARGES	37
AMAZONFEE	34
CRUK	16
DCGSSGIRL	13
DCGSSBOY	11
gift_0001_20	10
gift_0001_10	9

Figura 10 - Análise de consistência: Stock Codes fora do padrão de produtos.

Na dimensão de **unicidade**, foram identificadas 5.268 linhas totalmente duplicadas (0,97% do total), caracterizadas por valores idênticos em todos os atributos, incluindo o timestamp exato da venda, um padrão sem justificativa de negócio plausível (por exemplo, o mesmo item sendo "comprado" 20 vezes, em quantidade 1, no mesmo minuto, dentro do mesmo pedido). Esse problema foi tratado com a aplicação do método dropDuplicates() na construção da camada silver, antes dos demais filtros de limpeza já definidos, resultando na redução do total de linhas válidas de 397.884 para 392.692.

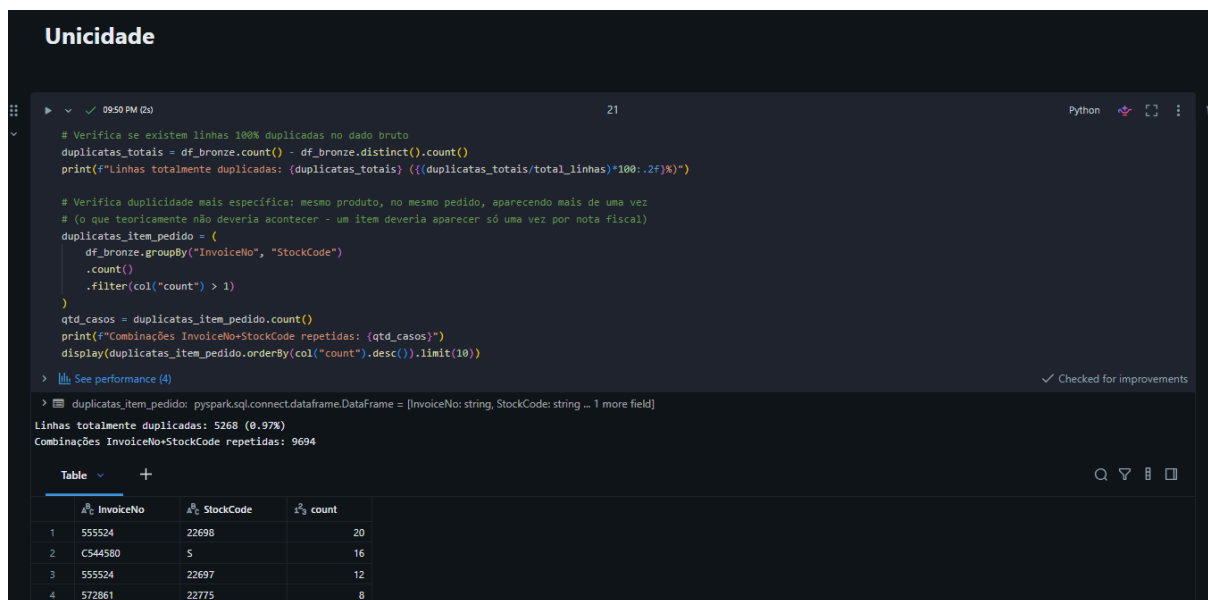


Figura 11 - Análise de unicidade

Na dimensão de **acurácia**, verificou-se a presença de valores de **UnitPrice** negativos (mínimo de -£11.062,06), tratados pelo filtro de exclusão de preços não positivos já aplicado na camada silver. Observou-se ainda que os maiores valores de **UnitPrice** do dataset concentram-se justamente nos códigos administrativos identificados na análise de consistência, reforçando que tais registros não representam transações de produtos. Um caso específico foi analisado em profundidade: uma venda de 80.995 unidades do produto de código 23843 (pedido 581483) foi cancelada 12 minutos depois (pedido C581484, com quantidade -80.995). Como o tratamento atual filtra apenas pelo sinal da quantidade, sem parear vendas aos seus cancelamentos correspondentes, essa venda específica permanece presente na camada silver mesmo tendo sido revertida.

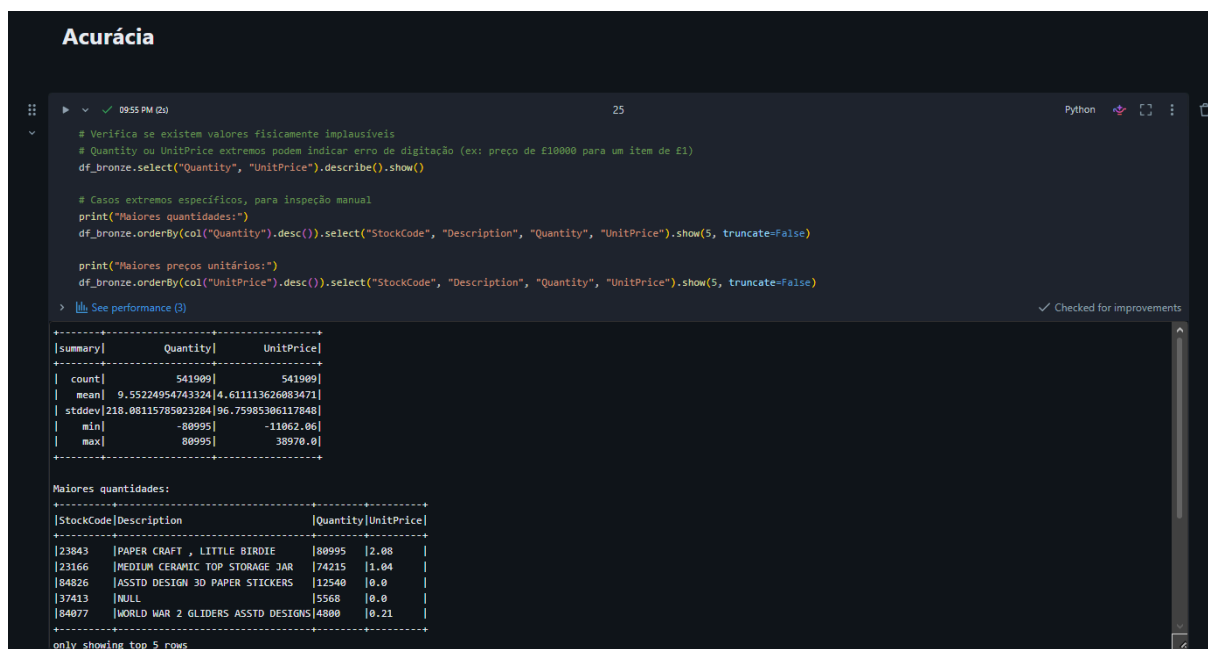


Figura 12 - Análise de acurácia

Na dimensão de **outliers**, utilizando o critério de três desvios-padrão acima da média, identificaram-se 542 registros (0,10%) com **Quantity** extrema e 1.001 registros (0,18%) com **UnitPrice** extremo. Notavelmente, 87,41% dos outliers de preço correspondem aos

mesmos códigos administrativos já identificados nas dimensões de consistência e acurácia, indicando que a maior parte dos valores estatisticamente extremos tem causa sistemática identificável, e não erro aleatório de digitação.

```
from pyspark.sql.functions import mean, stddev, abs as _abs

# Calcula média e desvio padrão de Quantity e UnitPrice (usando só valores positivos, já que negativos são outro problema já tratado)
stats = df_bronze.filter((col("Quantity") > 0) & (col("UnitPrice") > 0)).select(
    mean("Quantity").alias("media_qtd"), stddev("Quantity").alias("std_qtd"),
    mean("UnitPrice").alias("media_preco"), stddev("UnitPrice").alias("std_preco")
).collect()[0]

# Define outlier como valor a mais de 3 desvios-padrão da média (regra prática comum)
limite_qtd = stats["media_qtd"] + 3 * stats["std_qtd"]
limite_preco = stats["media_preco"] + 3 * stats["std_preco"]

print(f"Limite superior Quantity (média + 3 desvios): {limite_qtd:.2f}")
print(f"Limite superior UnitPrice (média + 3 desvios): {limite_preco:.2f}")

outliers_qtd = df_bronze.filter(col("Quantity") > limite_qtd).count()
outliers_preco = df_bronze.filter(col("UnitPrice") > limite_preco).count()

print(f"Registros com Quantity acima do limite: {outliers_qtd} ({(outliers_qtd/total_linhas)*100:.2f}%)")
print(f"Registros com UnitPrice acima do limite: {outliers_preco} ({(outliers_preco/total_linhas)*100:.2f}%)")

> See performance \(3\) ✓ Checked for improvements
```

Limite superior Quantity (média + 3 desvios): 477.11
Limite superior UnitPrice (média + 3 desvios): 111.65
Registros com Quantity acima do limite: 542 (0.18%)
Registros com UnitPrice acima do limite: 1001 (0.18%)

Figura 13 - Análise de outliers

Resumindo, a análise revelou dois problemas estruturais no conjunto de dados: duplicação de registros e sobreposição de códigos administrativos com códigos de produto, ambos com causa identificada e tratamento definido. A duplicação foi corrigida diretamente na camada silver. Já o uso de **StockCode** para lançamentos não-comerciais teve seu impacto financeiro quantificado em 1,69% do faturamento total da camada gold, e seu tratamento foi diferenciado por finalidade analítica: excluído da resposta relativa a produtos mais vendidos (onde distorceria diretamente o resultado), e mantido nas respostas de faturamento por região e por cliente (onde representa receita real associada à transação).

Análise Final dos Dados

Durante o processo de análise sob a ótica comercial, foi observado que o ticket médio tem um peso relevante na tomada de decisão dentro dos diferentes cenários estudados. Desta forma, ela foi incorporada nas tabelas respostas de todas as perguntas levantadas.

Pergunta 1: Quais são os produtos mais vendidos em quantidade?

Para responder essa questão, agrupou-se a tabela fato (fato_vendas) por **StockCode**, somando a quantidade vendida e o faturamento gerado, contando o número de pedidos distintos em que cada produto aparece, e calculando o ticket médio dos pedidos que contêm aquele item. Dois ajustes de qualidade foram aplicados antes da agregação: exclusão de códigos administrativos (que não representam produtos físicos, ex. "POST", "AMAZONFEE") e exclusão do pedido 581483, identificado na análise de qualidade como uma venda cancelada 12 minutos após sua realização.

```

4 minutes ago (7s) 32 Python
from pyspark.sql.functions import countDistinct

# Agrupa o fato por produto, excluindo:
# - códigos administrativos (não representam vendas de produto real)
# - o pedido cancelado 581483 (mesma correção aplicada nas demais tabelas gold)
top_produtos = (
    fato_vendas
    .filter(~col("StockCode").isin(codigos_ao_produto))
    .filter(~col("InvoiceNo").isin(["581483"]))
    .groupBy("StockCode")
    .agg(
        _sum("Quantity").alias("QuantidadeTotal"),      # soma de itens vendidos por produto
        _sum("TotalPrice").alias("FaturamentoTotal"),    # soma do valor gerado por produto
        countDistinct("InvoiceNo").alias("TotalPedidos") # em quantos pedidos distintos o produto aparece
    )
    # Ajuste de tipo: double -> decimal, evitando erro de arredondamento binário
    .withColumn("FaturamentoTotal", col("FaturamentoTotal").cast("decimal(12,2)"))
    # Ticket médio: valor médio do pedido completo, entre os pedidos que contêm este produto
    # (não é o preço do produto isolado, é o valor total do pedido em que ele aparece)
    .withColumn("TicketMedio", (col("FaturamentoTotal") / col("TotalPedidos")).cast("decimal(10,2)"))
    # Junta com dim_produto para trazer a descrição legível do produto (não só o código)
    .join(dim_produto, on="StockCode", how="left")
    .select("StockCode", "Description", "QuantidadeTotal", "FaturamentoTotal", "TotalPedidos", "TicketMedio")
    # Ordena pela quantidade, critério da pergunta 1 do MVP
    .orderBy(col("QuantidadeTotal").desc())
)

top_produtos.write.mode("overwrite").option("overwriteSchema", "true").saveAsTable("ecommerce_mvp.gold.top_produtos")
display(top_produtos.limit(10))

```

> [See performance \(2\)](#) ✓ Checked for improvements

Figura 14 - Código utilizado na pergunta 1

	A ^B StockCode	A ^B Description	1 ² QuantidadeTotal	.00 FaturamentoTotal	1 ² TotalPedidos	.00 TicketMedio
1	23166	MEDIUM CERAMIC TOP STORAGE JAR	77916	81416.73	195	417.52
2	84077	WORLD WAR 2 GLIDERS ASSTD DESIGNS	54319	13558.41	472	28.73
3	22197	POPCORN HOLDER	49160	37206.88	1035	35.95
4	850998	JUMBO BAG RED RETROSPOT	46078	85040.54	1600	53.15
5	85123A	WHITE HANGING HEART T-LIGHT HOLD...	36763	100547.45	1978	50.83
6	84879	ASSORTED COLOUR BIRD ORNAMENT	35263	56413.03	1375	41.03
7	21212	PACK OF 72 RETROSPOT CAKE CASES	33670	16381.88	1029	15.92
8	23084	RABBIT NIGHT LIGHT	27153	51251.24	801	63.98
9	22492	MINI PAINT SET VINTAGE	26076	16039.24	325	49.35
10	22616	PACK OF 12 LONDON TISSUES	25329	7261.77	382	19.01

Figura 15 - Tabela resposta da pergunta 1

É visto que o pote de cerâmica médio é o top 1 de faturamento e um dos maiores em volume também. Por ter um ticket médio bem mais alto que os demais do top 10, seu valor total em pedidos se destaca com mais facilidade.

Pode-se observar que o suporte branco (top 5 da lista) aparece em 10 vezes mais pedidos, porém seu ticket é proporcionalmente inferior ao do jarro, o que sugere uma compra mais pulverizada e recorrente.

Pergunta 2: Quais regiões concentram as vendas em volume e por faturamento?

Agrupou-se fato_vendas por **Country**, somando quantidade e faturamento, complementado por uma agregação de agg_pedidos (grão de pedido) para calcular ticket médio e total de pedidos por região — métrica adicional que enriquece a leitura de perfil de compra por mercado.

```

from pyspark.sql.functions import avg, count

# Exclui o pedido cancelado (mesma correção aplicada em top_produtos e top_clientes)
agg_pedidos = agg_pedidos.filter(col("InvoiceNo") != "581483")
agg_pedidos.write.mode("overwrite").option("overwriteSchema", "true").saveAsTable("ecommerce_mvp.gold.agg_pedidos")

# Recalcula ticket médio e total de pedidos por região com o dado corrigido
ticket_por_regiao = (
    agg_pedidos.groupBy("Country")
    .agg(
        _sum("ValorTotalPedido").alias("FaturamentoTotal"),
        count("InvoiceNo").alias("TotalPedidos"),
        avg("ValorTotalPedido").alias("TicketMedio")
    )
    .withColumn("FaturamentoTotal", col("FaturamentoTotal").cast("decimal(12,2)"))
    .withColumn("TicketMedio", col("TicketMedio").cast("decimal(10,2)"))
)

# Recria vendas_por_regiao a partir do fato (também sem o pedido cancelado) + ticket médio corrigido
vendas_por_regiao = (
    fato_vendas
    .filter(col("InvoiceNo") != "581483")
    .groupBy("Country")
    .agg(
        _sum("Quantity").alias("VolumeTotal"),
        _sum("TotalPrice").alias("FaturamentoTotal")
    )
    .withColumn("FaturamentoTotal", col("FaturamentoTotal").cast("decimal(12,2)"))
    .join(ticket_por_regiao.select("Country", "TotalPedidos", "TicketMedio"), on="Country", how="left")
    .orderBy(col("FaturamentoTotal").desc())
)

vendas_por_regiao.write.mode("overwrite").option("overwriteSchema", "true").saveAsTable("ecommerce_mvp.gold.vendas_por_regiao")
display(vendas_por_regiao.limit(10))

```

Figura 16 - Código utilizado na pergunta 2

	Country	VolumeTotal	FaturamentoTotal	TotalPedidos	TicketMedio
1	United Kingdom	4160310	7116555.04	16672	426.86
2	Netherlands	200361	285446.34	94	3036.66
3	EIRE	140133	265262.46	261	1016.33
4	Germany	119154	228678.40	457	500.39
5	France	111428	208934.31	390	535.73
6	Australia	83891	138453.81	57	2429.01
7	Spain	27933	61558.56	90	683.98
8	Switzerland	30082	56443.95	51	1106.74
9	Belgium	23237	41196.34	98	420.37
10	Sweden	36078	38367.83	36	1065.77

Figura 17 - Tabela resposta da pergunta 2

O Reino Unido concentra a maioria das vendas muito por conta do seu alto volume de pedidos, pois quando comparamos seu ticket médio com os demais países vemos que é o 2º menor da listagem dos top 10.

Holanda e Austrália têm bastante destaque no ticket médio, sendo países de interesse para buscar crescimento em volume de pedidos.

Pergunta 3: Quais são os maiores clientes em termos de faturamento?

Agrupou-se fato_vendas por CustomerID, somando o faturamento gerado por cada cliente, com o país associado obtido via first("Country") (validado previamente que cada cliente está associado a um único país nos dados). O mesmo filtro do pedido cancelado (581483) foi aplicado, por ser o mesmo cliente (16446) que aparecia artificialmente no topo do ranking.

```

from pyspark.sql.functions import first

# Agrupa agg_pedidos (grão de pedido) por cliente, para calcular ticket médio real por cliente
# agg_pedidos já está sem o pedido cancelado 581483 (correção aplicada anteriormente)
ticket_por_cliente = (
    agg_pedidos.groupBy("CustomerID")
    .agg(
        count("InvoiceNo").alias("TotalPedidos"), # quantos pedidos distintos o cliente fez
        avg("ValorTotalPedido").alias("TicketMedio") # valor médio dos pedidos do cliente
    )
    .withColumn("TicketMedio", col("TicketMedio").cast("decimal(10,2)"))
)

# Agrupa o fato por cliente, somando faturamento e trazendo o país associado
top_clientes = (
    fato_vendas
    .filter(~col("InvoiceNo").isin(["581483"])) # mesma correção do pedido cancelado
    .groupBy("CustomerID")
    .agg(
        _sum("TotalPrice").alias("FaturamentoTotal"),
        first("Country").alias("Country")
    )
    .withColumn("FaturamentoTotal", col("FaturamentoTotal").cast("decimal(12,2)"))
    # Junta com o ticket médio calculado a partir de agg_pedidos
    .join(ticket_por_cliente, on="CustomerID", how="left")
    .select("CustomerID", "Country", "FaturamentoTotal", "TotalPedidos", "TicketMedio")
    .orderBy(col("FaturamentoTotal").desc())
)

top_clientes.write.mode("overwrite").option("overwriteSchema", "true").saveAsTable("ecommerce_mvp.gold.top_clientes")
display(top_clientes.limit(10))

```

Figura 18 - Código utilizado na pergunta 3

	CustomerID	Country	FaturamentoTotal	TotalPedidos	TicketMedio
1	14646	Netherlands	280206.02	73	3838.44
2	18102	United Kingdom	259657.30	60	4327.62
3	17450	United Kingdom	194390.79	46	4225.89
4	14911	EIRE	143711.17	202	711.44
5	12415	Australia	124914.53	21	5948.31
6	14156	EIRE	117210.08	55	2131.09
7	17511	United Kingdom	91062.38	32	2845.70
8	16029	United Kingdom	80850.84	63	1283.35
9	12346	United Kingdom	77183.60	1	77183.60
10	16684	United Kingdom	66653.56	28	2380.48

Figura 19 - Tabela resposta da pergunta 3

O maior cliente em faturamento é da Holanda, onde foi visto anteriormente que é o país com o maior ticket médio.

Temos um outlier, no sentido de ser um pedido que ocorre numa frequência histórica baixa, na linha 9, onde há um único pedido avaliado em 77 mil libras aproximadamente, de um cliente do Reino Unido.

Já na linha 5 temos outro caso interessante, um cliente da Austrália com o maior ticket médio se desconsiderarmos o pedido do Reino Unido.

Avaliando as tabelas respostas, conseguimos montar um panorama comercial com dois perfis de operação coexistindo:

- Um mercado doméstico (Reino Unido) de alto volume, ticket médio moderado (426,86) e produtos de baixo valor unitário.
- Mercados internacionais (Holanda, Austrália, Irlanda) com poucos pedidos, mas ticket médio de 5 a 14 vezes maior, padrão comum de clientes atacadistas / varejistas, e não de consumo final.

Essa dualidade se confirma tanto por região quanto por cliente: os maiores clientes individuais concentram-se nesses mercados internacionais.

Além disso, a mesma lógica de concentração aparece dentro de produtos individuais: o produto líder em quantidade (Pote de cerâmica) tem 95% de seu volume vindo de um único pedido de um único cliente do Reino Unido, o mesmo cliente que lidera isoladamente o ranking com apenas essa compra. Isso mostra que compras pontuais de grande porte também ocorrem no mercado doméstico, não só nos internacionais.

Essa observação é relevante para o objetivo geral do projeto: rankings agregados podem mascarar se um resultado reflete demanda recorrente ou um evento pontual concentrado. Uma leitura estratégica robusta do panorama comercial deve considerar não só os totais, mas a distribuição desses totais entre poucos ou muitos eventos de compra.

Considerações finais

Pontos de melhoria

Um possível ponto de evolução seria a implementação de um pareamento sistemático entre vendas e cancelamentos. Ao longo da análise de qualidade, identificou-se um caso específico (**StockCode** 23843, pedido 581483) em que uma venda de grande volume foi cancelada minutos após sua realização, mas permaneceu presente na camada silver por o tratamento atual filtrar apenas pelo sinal da quantidade (**Quantity** > 0), sem verificar a existência de um cancelamento correspondente (identificado pelo prefixo "C" no **InvoiceNo**). O caso identificado foi tratado pontualmente nas tabelas gold que alimentam as respostas do MVP, mas uma solução genérica, que casasse automaticamente vendas com seus cancelamentos por cliente, produto, quantidade e proximidade temporal, não foi implementada, por representar uma complexidade de modelagem desproporcional ao tempo disponível para o projeto.

Outro ajuste desejável seria a automação do agendamento da pipeline. Na versão utilizada do Databricks (Free Edition), a execução dos notebooks foi feita manualmente a cada alteração, sem uso de Jobs agendados. Um ambiente com suporte a agendamento (como o Databricks disponível em contas pagas) permitiria configurar a reexecução automática do pipeline bronze-silver-gold sempre que novos dados fossem disponibilizados, além de possibilitar a integração direta com o repositório do projeto no GitHub, eliminando a necessidade de upload manual do arquivo CSV a cada nova ingestão.

Por fim, seria interessante aprofundar a análise de qualidade identificada nos **StockCodes** administrativos (POST, AMAZONFEE, BANK CHARGES, entre outros), avaliando a viabilidade de segregá-los sistematicamente em uma dimensão própria (ex.

`dim_tipo_lancamento`), em vez de tratá-los caso a caso conforme a pergunta de negócio. Essa estrutura permitiria consultas mais flexíveis, decidindo de forma explícita e parametrizável quando incluir ou excluir esses lançamentos, em vez de depender de listas de exclusão fixas no código, como foi feito neste projeto.

Autoavaliação

É sempre enriquecedor botar a mão na massa para colocar em prática o que é visto na teoria e os MVP's têm cumprido bem esse papel. Desenvolver o pipeline de dados desde o 0 ajudou a criar uma confiança de entendimento e replicabilidade, o que é bem positivo acadêmica e profissionalmente.

Foi percebido que mesmo em contextos mais abrangentes, como o do presente trabalho, é possível trazer valor e praticidade ao negócio através de um processo bem estruturado de banco de dados.

Ao final do projeto, foi possível responder todas as perguntas com perspectivas ainda mais descritivas do que inicialmente foi pensado, onde a métrica do ticket médio entrou na análise para apoiar insights, assim como a identificação do país de origem de cada cliente (pergunta 3).

Não foi possível concluir ou desenvolver, por decisão de escopo diante do cronograma de entrega e da complexidade/conhecimento exigido:

- Pareamento sistemático entre vendas e seus respectivos cancelamentos
- Segregação dos Stock Codes administrativos em uma dimensão própria

Já a ausência de bibliotecas de visualização (Matplotlib/Seaborn) e de agendamento automatizado da pipeline decorreram de limitações da versão gratuita do Databricks e do momento atual do curso (disciplinas de análise exploratória e visualização ainda não cursadas), e não de escolha de escopo.